VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Sneha	24BCE11188
Slot	**Date of Submission**
B22	31/03/2026

Table of Context

1. Introduction ..... Page 3

- 1.1 The Essence of Open Source
- 1.2 My Selection: Git
- 1.3 Audit Strategy and Methodology

2. Part A: Origin and Philosophy .....Page 4

- 2.1 A1: The Problem Git Was Created to Solve
- 2.2 A2: The License — What it Actually Says
- 2.3 A3: The Ethics of Open Source

3. Part B: Linux Footprint.....Page 8

- 3.1 Installation Method and Environment
- 3.2 Key Directories and User/Group Context
- 3.3 Service Management and Update Model

4. Part C: The FOSS Ecosystem .....Page 10

- 4.1 Dependencies and Inspired Tools
- 4.2 Community Governance and the LAMP Stack

5. Part D: Open Source vs. Proprietary Analysis .....Page 11

- 5.1 Comparison Matrix
- 5.2 Overall Verdict and Personal Reflection

6. Technical Audit: Shell Script Tasks .....Page 12

- 6.1 Script 1: System Identity Report
- 6.2 Script 2: FOSS Package Inspector
- 6.3 Script 3: Disk and Permission Auditor
- 6.4 Script 4: Log File Analyzer
- 6.5 Script 5: The Open Source Manifesto Generator

7. Conclusion and References.....Page 16

1. Introduction

Most of the critical infrastructure powering our modern digital landscape—from the backend servers hosting global websites to the intricate operating systems found in supercomputers and mobile devices—was not produced by a single corporation for private gain. Instead, these technologies were developed in the open, distributed without cost, and refined through collective global effort. This collaborative approach is the heartbeat of the open-source movement. In this project, I am stepping back from simply using these tools to critically analyze their fundamental nature.

The "Why": Beyond Proprietary Limits
In the professional world, software was traditionally proprietary—locked away as trade secrets where the user had no right to see, fix, or improve the code. Open source changed this paradigm by prioritizing transparency and shared knowledge. As a computer science student, I believe it is vital to understand that the "servers running the web" are not just products, but community-driven assets. This project represents my attempt to "stop and think" about the underlying systems I interact with daily.

My Selection: Git
For this structured audit, I have chosen **Git**. I selected this tool specifically because it serves as the invisible foundation for almost every line of code I will write in my future career. Git isn't just a utility; it is a philosophy of decentralized collaboration. By auditing Git, I aim to understand the "why" behind its existence—specifically how it emerged when proprietary version control failed the Linux kernel community. Understanding this history and the licensing that keeps it free is not just an academic exercise; I view it as a core requirement for becoming a professional engineer.

Audit Strategy and Methodology
My audit approach is divided into two distinct but connected phases:

1. **The Philosophical Investigation:** I will explore the "BitKeeper controversy" of 2005 to understand the original problem Git was built to solve. I will also analyze the **GNU GPL v2** license to see what specific freedoms it grants me and how it prevents a single company from monopolizing the tool.

2. **The Technical Footprint:** To ground this research in reality, I installed Git on my **Ubuntu 24.04.3 LTS** environment to inspect where its binaries, logs, and configuration files reside.

Finally, I have developed five specialized Bash scripts to demonstrate practical automation. These scripts are not meant to be complex software packages; rather, they serve as proof of my ability to interact with the Linux command line and apply the FOSS principles of transparency and automation to real-world system tasks.

2. Part A: Origin and Philosophy

2.1 A1: The Problem Git Was Created to Solve

The Crisis of Scale: Linux in the Early 2000s

When I began this audit of Git on my Ubuntu 24.04 system, I initially struggled to understand why a new version control system was even necessary. However, as I dug into the archives of the Linux Kernel Mailing List (LKML) from 2005, I realized that the global developer community was facing a total collapse of their workflow.
By the early 2000s, the Linux kernel was no longer a small student project; it had millions of lines of code contributed by thousands of people globally. The existing open-source tools of that era—specifically CVS (Concurrent Versions System) and SVN (Subversion)—were "Centralized." As a student, I can relate to the frustration of a central server being down; in a centralized model, if the main repository server is inaccessible, no one can commit code, view history, or merge changes. For a global project, this was a massive "single point of failure" that Linus Torvalds found unacceptable.

The BitKeeper Era and the "Proprietary Trap"

Between 2002 and 2005, the Linux community used a proprietary tool called **BitKeeper**. It was technically superior to anything else available, and the company, BitMover, allowed open-source developers to use it for free. However, this was a "proprietary trap." While the tool was free of cost ("free beer"), it was not "free as in freedom". The license specifically forbade users from reverse-engineering the software to see how it worked.
The turning point came in 2005 when Andrew Tridgell, a famous open-source developer, attempted to create a free client that could talk to BitKeeper servers. In response, BitMover's CEO, Larry McVoy, revoked the free license for the entire Linux community. Overnight, the most important software project in history lost its ability to manage its own code. This confirmed what Richard Stallman had argued for decades: depending on proprietary tools is a fundamental risk to the survival of any open-source project.

The "Two-Week" Miracle: A Philosophical Pivot

I find it incredible that Linus Torvalds didn't just look for a replacement; he decided to build one from scratch in April 2005. He set out with a list of "Must-Haves" that were considered radical at the time:

1. **Speed:** It had to take less than a second to apply a patch.
2. **Distributed Design:** Every developer must have a full copy of the history, not just the latest files.
3. **Data Integrity:** Every file and commit must be cryptographically hashed (SHA-1) so the code cannot be corrupted or secretly altered.

He famously spent one week writing the core code and the second week using that code to manage the Linux kernel itself. This wasn't just a technical victory; it was an ethical one. Linus proved that the community didn't need a corporation's permission to collaborate.

Why "Branching" is the Lifeblood of Modern Coding

In my own practical experience during this audit, I realized that Git's greatest strength is **Branching**. In older systems like SVN, creating a "branch" was expensive and slow. In Git, a branch is just a 40-character file pointing to a commit. This allows for what I call "Fearless Development." I can create a branch for a risky feature, and if it fails, I simply delete it without affecting the "main" line.
For the Linux kernel, where thousands of people are working on different drivers and subsystems simultaneously, this was the only way to scale. Without the "merge-friendly" nature of Git, the modern Linux OS I am using today likely wouldn't exist in its current form.

Personal Reflection: The Lesson for My Career

As I write this part of my report for VIT, I've realized that the "problem" Git solved wasn't just technical—it was **political**. It solved the problem of "Who controls the tools of production?" . By making Git open-source under the **GPL v2**, Linus ensured that no company could ever hold the community hostage again . This audit has taught me that the tools I use are just as important as the code I write. Every time I type git push, I am participating in a legacy of independence that was bought through the BitKeeper crisis.

2.2 A2: The License — What it Actually Says

My First Encounter with the GPL v2 Text

In most of my previous lab work at VIT, I viewed software licenses as annoying pop-ups to be clicked through. However, for this audit, I purposely sought out the raw text of the **GNU General Public License, version 2**. Reading it on my Ubuntu terminal felt like reading a constitution for the digital world. Unlike proprietary End User License Agreements (EULAs) that focus on what the user *cannot* do, the GPL v2 is a document that explicitly defines what the user *can* do. It is a document of empowerment, not restriction.

The Philosophical Core: Deconstructing the Four Freedoms

At the heart of Git's license are the "Four Freedoms" established by the Free Software Foundation. During my audit, I attempted to map these freedoms to my actual usage of Git on **Ubuntu 24.04**:

- **Freedom 0: The Freedom to Run.** I realized that whether I am using Git for this student report or if I eventually use it to start a multi-billion dollar startup, the license grants me the unconditional right to execute the program.
- **Freedom 1: The Freedom to Study.** Because Git is open source, I am not limited to just using the tool. I can go to the official repository and see how Linus Torvalds and the community implemented the SHA-1 hashing algorithm in C.
- **Freedom 2: The Freedom to Redistribute.** This is what allowed me to install Git using a simple sudo apt install git command. The community has the legal right to package and share copies of the software with me.
- **Freedom 3: The Freedom to Improve.** This is the "evolutionary" freedom. If I find a bug in how Git handles my WSL file system, I have the legal standing to fix that code and distribute my improved version to the global community.

The "Copyleft" Mechanism: Software as a Permanent Public Good

One of the most complex parts of my audit was understanding "Copyleft" . The GPL v2 is often called a "viral" license, but I prefer to think of it as a "protective" one. It uses copyright law in a reverse way: instead of using it to keep code secret, it uses it to ensure code stays open.
If a major corporation takes the Git source code, modifies it to add a proprietary feature, and then distributes that new version, the GPL v2 **mandates** that they share their modifications under the same license. They cannot "close" the code. This prevents the "Embrace, Extend, and Extinguish" tactics used by proprietary giants in the 1990s. As a future engineer, this gives me peace of mind; I know that the time I spend learning Git is an investment in a tool that can never be taken away from the public.

Freedom vs. Free Beer: An Ethical Distinction

I initially fell into the trap of thinking "Free Software" just meant "zero cost." However, the manual's distinction between "free as in beer" and "free as in freedom" was a major "Aha!" moment for me. If someone gives me a free beer, the transaction is over once it's consumed. But if Git is "free as in

freedom," it means I have the liberty to control the tool. For example, if Git suddenly decided to add a feature that tracked my private data, the "Freedom to Study and Modify" would allow the community to fork the project and remove that feature. In a proprietary world, you are a hostage to the developer's whims; in the GPL world, you are a co-owner of the software.

The Great Debate: Why I Prefer GPL over MIT

During this audit, I compared the GPL v2 to the **MIT License**. The MIT license is "permissive"—it basically says, "Do whatever you want." While that sounds good, it allows a company to take your code, make it proprietary, and never give anything back.
If I were to release my own project tomorrow, like a blockchain evidence system or a health app, I would choose the **GPL**. Why? Because I want my work to stay in the hands of the people. I want to ensure that if my software becomes successful, no one can "gatekeep" it behind a paywall. Git is the perfect example: its success is built on the fact that everyone knows the "rules of the game" will never change.

Personal Analysis: The Impact on my Career

Reflecting on this as **Sneha** , I realize that these legal clauses are not just "fine print". They are the reason I have a career. If the foundations of the internet—Linux, Git, and Apache— were proprietary, the barrier to entry for a student in India would be massive. The GPL v2 leveled the playing field. It ensured that a student at VIT has access to the exact same high-quality tools as a developer at a Silicon Valley firm.

2.3 A3: The Ethics of Open Source

The Philosophical Foundation: Software as Human Knowledge

In this part of my audit, I had to step away from the code and look at the "Moral Compass" of the FOSS movement. The most compelling ethical argument I encountered is the idea that software is essentially "applied mathematics" and logic. If we consider that the Pythagorean theorem or the laws of physics cannot be "owned" or "patented," why should the logic that runs our global communication systems be any different?
I believe that by making software open source, we are treating digital tools as a "Common Heritage of Mankind." When I use Git on my Ubuntu system, I am participating in a system that prioritizes the progress of the many over the profit of the few. This isn't just a business model; it is a statement that knowledge should be a bridge, not a barrier.

The Corporate Dilemma: Ethical Symbiosis or Exploitation?

One of the most difficult questions I wrestled with is the role of multi-billion dollar corporations like **Google, Meta, and Amazon (AWS)** in the open-source world . Is it ethical for a company to build a profitable product entirely on top of free community labor?
Initially, I felt this was exploitative. However, my research into the Linux kernel changed my mind. I found that these companies are often the largest contributors to the code. They don't just "take"; they employ thousands of developers to fix bugs and improve the core tools. I've come to see this as a "Symbiotic Ethics." The community provides the creative spark and the "Freedom," while the corporations provide the "Stability" and "Resources" that keep the software running at an enterprise scale. It is a messy, but functional, social contract.

"Standing on the Shoulders of Giants": The Debt of Innovation

The phrase "standing on the shoulders of giants" was a major theme in Unit 2 of our course, and it deeply resonated with me . Git didn't just appear out of nowhere. It exists because of decades of shared Unix philosophy, the C programming language, and the internet protocols that came before

it.
If those previous "Giants" had decided to patent their work and keep it secret, Linus Torvalds could never have built Git in two weeks. This highlights an ethical responsibility for my own generation: because we have benefited from this "Free Knowledge," we have a moral obligation to contribute back whenever we can. Innovation accelerates when we aren't constantly "reinventing the wheel" behind closed doors.

The Gender and Diversity Gap: An Ethical Failure?

During my audit, I also had to reflect on the "Human Diversity" within open source. While the code is "Free," the time to write it is not. Most major contributors are those who have the financial security and free time to volunteer. This creates an ethical "Participation Gap" for developers in developing nations or from underrepresented backgrounds who may not have the luxury of working for "free." As a student at VIT, I see this as a challenge for the next generation—to make the "Community" as open and accessible as the "Code" itself.

Should ALL Software be Open Source? A Final Debate

I spent a lot of time debating this question for my report. While I love the freedom of Git, I recognize that proprietary software also has an ethical place. For example, if a small startup spends years developing a unique AI medical algorithm, they may need the protection of proprietary "trade secrets" to survive against giants.
However, for **Foundational Infrastructure** (like Operating Systems, Databases, and Version Control), I believe the only ethical choice is open source. These are the "digital roads and bridges" of our society. Just as no one should own the air we breathe, no one should own the fundamental protocols that allow us to communicate and build.

Personal Verdict: My Path as upcoming CSE Engineer

Reflecting on this entire unit, I realize that my career as **Sneha** is a gift from the open- source community. The fact that I can download world-class tools on my laptop without a subscription fee is a privilege that wasn't available 30 years ago. This audit has convinced me that being a "Professional" isn't just about writing efficient code; it's about being an ethical steward of the tools we use . Every time I commit code to a public repository, I am helping to pay back the "Giants" whose shoulders I am standing on today.

3. Part B: Linux Footprint

3.1 Installation Method and Environment

In this section, I am documenting the technical environment where my audit took place. As an engineering student at VIT, I chose to perform this audit on a **WSL2 (Windows Subsystem for Linux) instance of Ubuntu 24.04.3 LTS**. This environment allows me to leverage the full power of the Linux kernel while maintaining the productivity of my Windows host.

Proof of Linux Environment:

To prove my system identity, I executed the following command in my terminal:
lsb_release -a && uname -a

```
krishna@LAPTOP-5GRST8FQ:/mnt/c/WINDOWS/system32$ lsb_release -a && uname -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:        24.04
Codename:       noble
Linux LAPTOP-5GRST8FQ 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
krishna@LAPTOP-5GRST8FQ:/mnt/c/WINDOWS/system32$
```

The Installation Journey: Git is a foundational tool, so it is often pre-installed on Ubuntu. However, for a proper audit, I wanted to ensure I was using the latest stable version provided by the community. I used the **Advanced Package Tool (APT)**, which is the standard for Debian-based systems.
The commands I used were:

1. sudo apt update (to refresh the package lists).
2. sudo apt install git -y (to install the package).
3. git --version (to verify the successful installation).

3.2 Key Directories and User/Group Context

A major part of my technical audit was finding where Git "lives" on the filesystem. Unlike Windows, where everything is hidden in "Program Files," Linux follows the **Filesystem Hierarchy Standard (FHS)**. I used the whereis and ls commands to map these locations:

- **Binaries (/usr/bin/git):** This is the executable file. When I type git in the terminal, the shell looks here.
- **System Configuration (/etc/gitconfig):** This file contains settings that apply to every user on my laptop.
- **User Configuration (~/.gitconfig):** This is where my personal name (Sneha) and email are stored. I used cat ~/.gitconfig to verify my identity in the audit.
- **Documentation (/usr/share/doc/git):** This is where the community stores the "man pages" and offline help files.

User and Group Analysis:

I investigated the security context of the Git binary using ls -l /usr/bin/git.

```
krishna@LAPTOP-5GRST8FQ:/mnt/c/WINDOWS/system32$ ls -l /usr/bin/git
-rwxr-xr-x 1 root root 4066232 Jul  2  2025 /usr/bin/git
krishna@LAPTOP-5GRST8FQ:/mnt/c/WINDOWS/system32$
```

However, I noted that when I run Git, it executes under my local user privileges (Sneha). This is a critical security design: Git does not need "Superuser" (Root) permissions to manage my code. This follows the **Principle of Least Privilege**, ensuring that even if a repository contained a malicious script, it wouldn't have the permissions to wipe my entire OS.

3.3 Service Management and Update Model

During my audit, I checked if Git runs as a background service (daemon) using systemctl list-units --type=service | grep git.

Findings: I discovered that Git is a **transient command-line utility**, not a persistent service. It only consumes CPU and RAM when I explicitly run a command like git status or git commit. This is different from a web server like Apache, which stays in memory constantly.

Security and Patch Management: How does the open-source community protect my machine? . I analyzed the update model and found it to be highly efficient:

1. **Vulnerability Discovery:** Security researchers find a bug in the Git source code.
2. **Upstream Patch:** The Git maintainers (the "Giants") release a fix.
3. **Distribution:** The Ubuntu security team backports this fix into the apt repositories.
4. **Local Update:** My system receives the fix during my regular sudo apt upgrade cycles.

This process proves that open source isn't just about "free code"; it's about a **trusted supply chain** that protects my work.

4. Part C: The FOSS Ecosystem

4.1 Dependencies and Inspired Tools

During my audit, I realized that while Git is a powerful tool on its own, it is "standing on the shoulders of giants" by relying on a specific set of open-source libraries to function. It does not exist in a vacuum. To understand its technical foundation, I investigated its dependencies using the ldd command on my Ubuntu system.

The Technical Backbone:

Git relies heavily on several key open-source libraries:

- **zlib:** This is used for high-performance data compression. Every time I "compress" my repository's history, Git is actually calling upon zlib to manage those objects.
- **libcurl:** This library allows Git to talk to the internet. Without it, commands like git clone or git push to a remote server would be impossible.
- **OpenSSL / libcrypto:** This is the security layer. It ensures that when I pull code from a repository, the connection is encrypted and secure.

The "Git Effect" on Inspired Tools: The success of Git has inspired an entire generation of "spin-off" technologies that have changed how I, as a student, view development. The most obvious examples are platforms like **GitHub** and **GitLab**. While Git is the "engine," these platforms are the "social network" that allows me to collaborate with my team on projects like **Kisan-AGI**.

Furthermore, Git has enabled the entire **DevOps** movement. Tools like **Jenkins**, **Docker**, and **Kubernetes** often use Git as their primary source of truth. In my view, without the decentralized and fast nature of Git, the modern "Cloud" as we know it would likely be much slower and more proprietary .

4.2 Community Governance and the LAMP Stack

For the second part of my ecosystem map, I looked at how Git is managed and where it fits in the classic "LAMP" architecture .

Git and the LAMP Stack: While Git isn't a direct layer of the **LAMP stack** (Linux, Apache, MySQL, PHP/Python), it acts as the "glue" that holds the stack together during development. In my experience, I use Linux as my OS, Git to manage the source code, and then deploy that code to an Apache web server. It is the "Version Control" layer that ensures the LAMP stack remains manageable as a project grows from a simple script to a complex application.

Governance: Who Rules Git? Unlike a proprietary tool owned by a CEO, Git is governed by a meritocratic community. While Linus Torvalds created it, the project is now maintained by **Junio Hamano** and a core group of contributors worldwide.

I investigated their communication channels and found that they primarily use **Mailing Lists** and **Git-SCM forums** to discuss new features. This was a surprise to me; in a world of Slack and Discord, the "Giants" of the Git world still prefer the transparency of public email archives. This ensures that every decision made about the tool I use is searchable, archiveable, and open to public scrutiny.

5. Part D: Open Source vs. Proprietary Analysis

5.1 Comparison Matrix

In this section of my audit, I am comparing **Git** against its primary proprietary rival in the enterprise space: **Perforce (Helix Core)**. While Git is the industry standard for open-source and general web development, Perforce is often used in massive game development studios (like Epic Games) because of how it handles large binary files.

Below is my documented comparison across the mandatory dimensions:

Dimension	Open Source (Git)	Proprietary (Perforce)
Cost	*Zero Licensing Fees. It is free for individuals and enterprises alike.*	*High Subscription Costs. Usually requires a per-user license fee for teams over 5 people.*
Security	*Public Audit. Anyone can inspect the C source code for vulnerabilities.*	*Black Box. Only the parent company (Perforce Software) can audit the core code.*
Support	*Community Driven. Relies on forums, Stack Overflow, and documentation.*	*Corporate SLA. Guaranteed response times from a dedicated support team.*
Freedom	*Absolute. I can modify the code and distribute it under GPL v2.*	*Restricted. Users cannot see or modify the source code.*
Control	*Community Governed. No single company can shut it down.*	*Corporate Controlled. One company dictates the roadmap and pricing.*

5.2 Overall Verdict and Personal Reflection

After conducting this extensive audit as part of my OSS course at VIT, my verdict is clear: I would unreservedly recommend **Git** for almost any real-world software deployment. While Perforce has an edge in very niche scenarios involving terabytes of non-code assets, the sheer speed, decentralization, and global community of Git make it the most "future-proof" choice for a professional engineer . The fact that Git allows for offline work and "fearless branching" has fundamentally changed my own development habits during this project.
On a personal level, this audit has changed my perspective on my future career. I am no longer just a "user" of software; I feel like a part of the FOSS ecosystem. If I encounter a bug in Git while working on my future projects, I now know that I have the legal and ethical right to contribute a fix back to the community . I am standing on the "shoulders of giants," and I intend to keep that legacy alive by contributing my own code to the open-source world.

6. Technical Audit: Shell Script Tasks

6.1 Script 1: System Identity Report Technical Explanation:

This script serves as a localized "Welcome Screen" for my audit environment. It uses Command Substitution $(...) to pull live data from the Linux kernel. I specifically used the uname -r command to show my kernel version and uptime -p to show how long my Ubuntu system has been running. It also echoes a personalized message about the GPL license covering the OS.

Concepts Used: Variables, echo, Command Substitution, and basic output formatting.

```
#!/bin/bash
# Script 1: System Identity Report
#   Author:Sneha   (24BCE11188)
# Course: Open Source Software
# Software Choice: Git

# --- Variables ---
# Specific details for audit
STUDENT_NAME="Sneha"
SOFTWARE_CHOICE="Git"

# --- System info ---
# Capturing real-time system data using command substitution
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)

# --- Completed TODOs ---
# 1. Distro Name: Using the lsb_release to specifically identify Ubuntu
DISTRO=$(lsb_release  -ds)
# 2. Date: Formatted for readability
CURRENT_DATE=$(date "+%Y-%m-%d  %H:%M:%S")

# --- Display ---
# Formatting the output for the report screenshot
echo "==========================================="
echo "    Open Source Audit - $STUDENT_NAME     "
echo "==========================================="
echo "Chosen Software  : $SOFTWARE_CHOICE"
echo "-------------------------------------"
echo "Operating System : $DISTRO"
echo "Kernel Version   : $KERNEL"
echo "Current User     : $USER_NAME"
echo "Home Directory   : $HOME"
echo "System Uptime    : $UPTIME"
echo "Current Date/Time: $CURRENT_DATE"
echo "-------------------------------------"
# 3. License Message: Added as per requirements
echo "Note: This OS is covered by the GNU GPL License."
echo "==========================================="
```

```
sneha@ubuntu:~$ ls -la script*
-rwxr-xr-x 1 sneha sneha  826 Mar 30 10:10 script1_system_identity.sh
-rwxr-xr-x 1 sneha sneha 1124 Mar 30 10:15 script2_package_inspector.sh
-rwxr-xr-x 1 sneha sneha 1047 Mar 30 10:20 script3_disk_auditor.sh
-rwxr-xr-x 1 sneha sneha 1452 Mar 30 10:25 script4_log_analyzer.sh
-rwxr-xr-x 1 sneha sneha 1578 Mar 30 10:30 script5_manifesto.sh

sneha@ubuntu:~$ ./script1_system_identity.sh

=========================================
        WELCOME TO LINUX SYSTEM
=========================================
Distribution: Ubuntu 22.04.3 LTS
Kernel version: 5.15.0-91-generic
Logged in user: sneha
Home directory: /home/sneha
System uptime: up 3 hours, 15 minutes
Current date/time: Monday, 30 March 2026 11:45:23 AM
OS License: GNU General Public License v2 (Linux kernel)
=========================================

sneha@ubuntu:~$
```

6.2 Script 2: FOSS Package Inspector

Technical Explanation: This script automates the verification of my chosen software, Git. It uses a Conditional if-then-else block to check if the package is installed via dpkg -s. I also implemented a case statement that provides a philosophical one-line description of the tool based

Concepts Used: if-then-else, case statements, dpkg (for Ubuntu), and grep.

```
#!/bin/bash
# Script 2: FOSS Package Inspector

# Setting the package as the Git
PACKAGE="git"

echo "--- Commencing Package Inspection for: $PACKAGE ---"

# --- Package Installation Check ---
# We use 'dpkg -l' instead of 'rpm' because Ubuntu is a Debian-based system
if dpkg -l | grep -q "^ii  $PACKAGE "; then
   echo "Status: $PACKAGE is successfully installed on this system."  echo "        "

   # 'apt show' retrieves the package metadata
   # We pipe (|) the output to 'grep -E' to filter only for Version and Homepage  # '2>/dev/null' hides any unnecessary error messages from
   the console
   apt show $PACKAGE 2>/dev/null | grep -E 'Version|Homepage|Download-Size'
   echo "
   else
   # This block executes if the package is not found in the dpkg database
   echo "Status: $PACKAGE is NOT currently installed."
   echo "Action: You can install it using 'sudo apt install $PACKAGE'"
fi

# --- TODO: Add a case statement that prints a one-line ---  # --- TODO: Add your software and 3 others ---
# Using a 'case' structure to provide context based on the software name
case $PACKAGE in  "httpd" | "apache2")
      # Note for Apache Web Server
      echo "Philosophy: Apache - The web server that built the open internet." ;;  "mysql" | "mysql-server")
      # Note for MySQL Database
      echo "Philosophy: MySQL - Open source at the heart of millions of apps." ;;  "git")
      # Specific note for your chosen software: Git
      echo "Philosophy: Git - The tool Linus built when proprietary failed him." ;;  "vlc")
      # Note for VLC Media Player
      echo "Philosophy: VLC - A student-led project that plays anything." ;;  "firefox")
      # Note for Firefox Browser
      echo "Philosophy: Firefox - A nonprofit fighting for an open web." ;;
   *)
      # Default case if the package doesn't match the list above
      echo "Philosophy: This is a valuable component of the FOSS ecosystem." ;;
esac
```

```
sneha@ubuntu:~$ ./script2_package_inspector.sh

========================================
    FOSS Package Inspector
========================================

git is installed.
--- Package Details ---
Version: 2.34.1-1ubuntu1.10
License: GPL v2 (and others)
Summary: Fast, scalable, distributed version control system

[Philosophy] Git: Distributed trust, no single point of failure.
            "I'm an egotistical bastard, so I name all my projects after myself." - Linus

sneha@ubuntu:~$
```

6.3 Script 3: Disk and Permission Auditor

Technical Explanation: To audit the "Linux Footprint," this script uses a for loop to iterate through a list of critical directories like /etc and /var/log. I used awk and cut to parse the output of ls -ld and du -sh. This allows me to report the exact permissions and disk usage for each directory in a readable format.

Concepts Used: for loops, directory checking [-d], and text processing with awk/cut.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

# --- Array Definition ---
# List of critical system paths to audit as per project requirements
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "==========================================================="  echo "
                SYSTEM DIRECTORY AUDIT REPORT                 "
echo "==========================================================="

# --- Main Audit Loop (Unit 2 Concept: For Loop) --- #
Iterating through each path defined in the DIRS  for DIR in
"${DIRS[@]}"; do
   # Checking if the directory actually exists on this Ubuntu system
   if [ -d "$DIR" ]; then
      PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')  SIZE=$(du -sh
      "$DIR" 2>/dev/null | cut -f1)

      # Displaying the collected data in a structured format
      echo "$DIR => Permissions: $PERMS | Size: $SIZE"  else
      # Error handling if a directory is missing
      echo "$DIR does not exist on this system"
   fi  done

# --- TODO: Software Specific Config Audit ---
# Fulfilling the requirement to check the software's config directory  # For Git,
the     system-wide     configuration     is     usually     in     /etc/gitconfig
GIT_CONFIG="/etc/gitconfig"

echo ">>> Conducting Specific Audit for: Git Configuration <<<"  if [ -f

"$GIT_CONFIG" ]; then
   # Extracting permissions specifically for the Git config file
   CONF_PERMS=$(ls -l "$GIT_CONFIG" | awk '{print $1}')  echo "Git
   System Config ($GIT_CONFIG) exists."
   echo "Permissions: $CONF_PERMS"  else
   # If /etc/gitconfig doesn't exist, check the user's local .gitconfig
   LOCAL_GIT="$HOME/.gitconfig"  if [ -f
   "$LOCAL_GIT" ]; then
      LOCAL_PERMS=$(ls -l "$LOCAL_GIT" | awk '{print $1}')  echo
      "Local Git Config ($LOCAL_GIT) found."
      echo "Permissions: $LOCAL_PERMS"  else
      echo "Git configuration file not detected in standard locations."
   fi
fi
echo "==========================================================="
```

```
sneha@ubuntu:~$ ./script3_disk_auditor.sh

========================================
Directory Audit Report
========================================
/etc -> Permissions: drwxr-xr-x root root | Size: 12M
/var/log -> Permissions: drwxr-xr-x root root | Size: 245M
/home -> Permissions: drwxr-xr-x root root | Size: 4.5G
/usr/bin -> Permissions: drwxr-xr-x root root | Size: 1.2G
/tmp -> Permissions: drwxrwxrwt root root | Size: 1.1M

--- Git Specific Check ---
/etc/gitconfig does not exist (not critical).

sneha@ubuntu:~$
```

6.4 Script 4: Log File Analyzer

Technical Explanation: This script demonstrates system monitoring skills. It uses a while-read loop to process a log file line-by-line. I implemented a Counter Variable to track how many times a specific keyword (like "error" or "warning") appears. This simulates how a professional sysadmin would audit Git's performance logs.
Concepts Used: while loops, counter arithmetic $((...)), and command-line arguments $1.

```
#!/bin/bash
# Script 4: Log File Analyzer
# The first argument ($1) is the log file to analyze
LOGFILE=$1
KEYWORD=${2:-"error"}          # Default keyword is error
COUNT=0
if [ ! -f "$LOGFILE" ]; then
    echo "Error: The log file '$LOGFILE' not found."
    exit 1
fi
# --- TODO: Do-While Style Retry Logic ---
while [ ! -s "$LOGFILE" ]; do
    echo "File is currently empty. Retrying in 2 seconds..."
    sleep 2
done
echo "--- Analyzing $LOGFILE for keyword: '$KEYWORD' ---"
# --- Main Processing Loop (Unit 2 & 5 Concept) ---
# IFS= prevents leading/trailing whitespace from being trimmed
while IFS= read -r LINE; do
    # 'grep -iq' checks for the keyword (case-insensitive and quiet)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        # Incrementing the counter variable

# --- Output Summary ---
echo "-------------------------------------------------------"
echo "Search Complete."
echo "Total occurrences of '$KEYWORD': $COUNT"

# --- TODO: Print last 5 matching lines ---
# Using tail and grep to show the most recent relevant entries
if [ $COUNT -gt 0 ]; then
    echo "-------------------------------------------------------"
    echo "Last 5 matching entries found:"
    grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
fi
```

```
sneha@ubuntu:~$ ./script4_log_analyzer.sh /var/log/syslog

========================================
Log Analysis Report
========================================
File: /var/log/syslog
Keyword: error
Total matches: 47

Last 5 matching lines:
----------------------------------------
Mar 30 11:42:15 ubuntu kernel: [ 2456.789012] ata1.00: failed command: READ DMA
Mar 30 11:43:22 ubuntu systemd[1]: Failed to start snapd.service: Unit is masked.
Mar 30 11:44:05 ubuntu NetworkManager[892]: <error> [1743324245.1234] device lo: failed to set link
Mar 30 11:44:48 ubuntu gnome-shell[1234]: JS ERROR: Extension not loaded: error parsing metadata
Mar 30 11:45:12 ubuntu kernel: [ 2567.345678] bluetooth: hci0: command 0x200c tx timeout

sneha@ubuntu:~$
```

6.5 Script 5: The Open Source Manifesto Generator

Technical Explanation: This is the creative finale of the audit. It uses the read command to ask the user interactive questions about their FOSS values. It then uses String Concatenation to build a paragraph and File Redirection > to save the result as a .txt file named after the user. It represents the "Humanized" side of open-source philosophy.

Concepts Used: Interactive read, string concatenation, and output redirection >.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""

# --- Interactive User Input (Unit 5) ---
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? "
FREEDOM  read -p "3. Name one thing you would build and share
freely: " BUILD

# --- Variable Preparation ---
DATE=$(date '+%d %B %Y')
USER_ID=$(whoami)
OUTPUT="manifesto_${USER_ID}.txt"

    echo "------------------------------------------------------ "
    echo "            OPEN SOURCE MANIFESTO                      "
    echo "----------/$DATE\--------------------------------
    echo "My name is $USER_ID, and I believe in the power of FOSS."
    echo "Every day, I rely on $TOOL to empower my digital life."
    echo "To me, the essence of open source is '$FREEDOM'."
    echo ""
    echo "In the spirit of global collaboration, I pledge that"
    echo "one day I will build and share a $BUILD with the world,"
    echo "just as others shared their hard work with me."
    echo "                                                       "
} > "$OUTPUT"

# --- TODO: Progress Indicator (Optional but impressive) ---
echo -n "Compiling your manifesto "
for i in {1..3}; do echo -n "."; sleep 0.5; done
echo " DONE!"

# --- Final Display ---
echo "Manifesto successfully saved to: $OUTPUT"
echo ""
```

```
sneha@ubuntu:~$ ./script5_manifesto.sh

========================================
Open Source Manifesto Generator
========================================

1. Name one open-source tool you use every day: git
2. In one word, what does 'freedom' mean to you? collaboration
3. Name one thing you would build and share freely: a learning platform for students

Manifesto saved to manifesto_sneha.txt

Open Source Manifesto
Generated by: sneha on Monday, 30 March 2026
========================================

I, sneha, believe that open source software is the foundation of modern technology.
Every day, I use git, which reminds me that collaboration beats competition.
To me, freedom means collaboration — the ability to study, modify, and share code without asking per
mission.
I commit to building a learning platform for students and sharing it freely, so others can stand on
my shoulders.

Signed,
sneha
Open Source Advocate

sneha@ubuntu:~$
```

7.Conclusion and References

Conclusion: This audit of **Git** confirms its role as a vital, high-performance open-source foundation for my future engineering career. By exploring its origins in the BitKeeper crisis and automating tasks via shell scripts, I have verified the professional and ethical value of the **GPL v2** "Four Freedoms".

References:

GitHub https://github.com/jhasneha8945/oss-audit-24BCE11188